

Laboration 5 – Integration från sensor till konsument

Uppgift och förutsättningar

Koppla en MQTT-broker till ett HTTP-API genom en adapter. Använd C++-demon eller motsvarande komponenter som du redan har byggt. Git är valfritt i denna fristående laboration. I det efterföljande IoT-caset används gruppens befintliga Git-repository.

Avsätt ungefär 60 minuter för referensflödet och fortsätt sedan med [IoT-caset](#). Du behöver C++17, CMake och Mosquitto med kommandoradsklienterna. [Demots README](#) innehåller alla bygg- och körkommandon för Windows och Linux.

Del 1 – Förutsäg flödet

Rita sensor, broker, adapter, API och konsument. Skriv vid varje pil vem som startar anslutningen, vilket protokoll som används och vilken port som tar emot trafiken. Använd TCP 1885 för MQTT och TCP 8085 för API:t i referensmiljön.

Svara innan du kör: kommer konsumenten att ansluta till sensorn, brokern eller API:t?

Del 2 – Bygg och kontrollera kontraktet

1. Bygg enligt demots README och kör `ctest`
2. Läs `reading.hpp` och hitta anropen till `validate`
3. Generera `reading.json` med sensorprogrammet
4. Kontrollera att `sensorId`, `value` och `unit` finns i filen
5. Förklara varför ett orimligt internt värde ska stoppas redan före sändning

Referenskontraktet använder JSON, temperatur i C och intervallet -50 till 100 inklusive gränserna. Okända extra fält ignoreras i demon. I caset gäller ert dokumenterade format och dess regler.

Del 3 – MQTT och API

1. Starta lokal broker och API i separata terminaler
2. Kör konsumenten och anteckna resultatet innan första mätningen
3. Starta MQTT-mottagaren och publicera därefter `reading.json`
4. Kontrollera att mottagaren har avslutats och att `received.json` innehåller datan
5. Kör adaptern med den mottagna filen
6. Kör konsumenten och jämför värdet med sensorns ursprungliga värde

Mottagaren tar ett meddelande per körning. Starta den igen inför varje ny publicering. Ett terminalfönster med gammal output bevisar inte att ett nytt meddelande kom fram.

Del 4 – Testa gränserna

Fyll i en kopia av [testprotokollet](#). Spara kommando, förväntan, observerat resultat och slutsats.

Test	Gör så här	Det du undersöker
Giltig data	Kör hela flödet	Samma värde hos konsumenten
Fel datatyp	Kör adaptern med <code>testdata/invalid.json</code>	Validering före API-anrop
Direkt felaktigt POST	Använd curl-kommandot i demots README	API:ts egen validering
API avstängt	Stoppa API:t och kör adaptern	Transportfel och timeout
API omstartat	Kör konsumenten före ny POST	Minneslagrets livslängd
Fel topic	Publicera till <code>iot25/dag5/other</code>	Mottagarens prenumeration

Avsluta en väntande MQTT-mottagare med Ctrl+C eller låt dess 15-sekunderstimerout löpa ut. Skicka ny giltig data efter feltesterna och bekräfta att flödet fungerar igen.

Del 5 – Förklara resultatet

- Beskriv skillnaden mellan ett valideringsfel, ett HTTP-felsvar och ett transportfel
- Markera vilken komponent som äger den senaste mätningen
- Beskriv en fördel och en kostnad med att brokern och API:t är separata tjänster
- Skriv vad som behöver ändras om tjänsterna placeras på olika datorer

Del 6 – Valbar fördjupning: JSON via MQTT och XML till REST-API:t

När grundflödet fungerar kan ni prova att använda olika representationer på de två kommunikationssträckorna. Sensorn skickar JSON via MQTT som tidigare, men adaptern omvandlar mätningen till XML innan den skickas till REST-API:t. Detta är vanligt när en IoT-lösning behöver integreras med ett system som har ett annat datakontrakt.

```

Sensor → JSON via MQTT → broker → subscriber → mottagen JSON-fil
                                     ↓
                               Adapter: pars och validera JSON
                                     ↓
                               Intern Reading
                                     ↓
                               Validera och serialisera XML
                                     ↓
                               HTTP POST med application/xml
                                     ↓
                               API: pars och validera XML

```

Referensdemon och elevpaketet använder JSON på båda sträckorna. Den här delen kräver att ni ändrar både adaptern och API:t. Att bara byta **Content-Type** omvandlar inte innehållet. Det oförändrade API:t svarar HTTP 415 om ni skickar XML till det.

Samma mätning i två format

Behåll JSON-filen från grundflödet på MQTT-sträckan:

```
{
  "sensorId": "temp-01",
  "value": 21.7,
  "unit": "C"
}
```

Låt adaptern skapa följande XML som body till `POST /api/readings`:

```
<reading>
  <sensorId>temp-01</sensorId>
  <value>21.7</value>
  <unit>C</unit>
</reading>
```

Fältnamn, enhet och temperaturintervall behåller samma betydelse. REST-API:er kan använda XML som representation. XML i en HTTP-body gör inte i sig gränssnittet till SOAP.

Arbetsgång

1. Behåll sensorprogrammet, MQTT-topic och mottagningen till `received.json` från grundflödet
2. Återanvänd adapters JSON-parser för att bygga och validera en intern `Reading`
3. Lägg till en separat funktion som validerar `Reading` och serialiserar den till XML med ett XML-bibliotek från dag 4, exempelvis pugixml
4. Låt adaptern skicka XML-texten med `Content-Type: application/xml` till samma POST-endpoint
5. Utöka API:t så att det väljer XML-parser när innehållstypen är `application/xml` och behåller JSON-parsern för `application/json`
6. Parsa XML till samma interna `Reading` och kontrollera obligatoriska element, talformat, enhet och temperaturintervall innan lagret ändras
7. Behåll API:ts svar och GET-representation som JSON så att den befintliga konsumenten kan användas utan ändring
8. Bygg om adapter och API, stoppa den gamla API-processen och starta den nya versionen innan ni testar

Använd XML-biblioteket för att skapa element och textvärden så att exempelvis `&` och `<` får korrekt escaping. Bygg inte XML genom oskyddad strängkonkatenering. Säkerställ att talet skrivs och läses med decimalpunkt och att hela taltexten kontrolleras, så att exempelvis `21.7abc` avvisas.

XML-biblioteket ingår inte i dag 5-paketet. Återanvänd beroendet och byggkonfigurationen från dag 4 eller lägg till det i ert eget CMake-projekt. JSON- och XML-funktionerna ska använda samma interna datamodell och valideringsregler.

Testa och jämför

Starta broker och det uppdaterade API:t. Starta sedan MQTT-mottagaren före publiceringen, publicera `reading.json` och invänta att `received.json` har tagits emot. Kör den uppdaterade adaptern och därefter konsumenten. Visa gärna XML-texten i adapters logg eller spara den i `reading.xml` före sändning.

Test	Indata eller åtgärd	Förväntat resultat
Hela flödet	Giltig JSON via MQTT, XML från adapter till API	HTTP 201 och samma mätning hos konsumenten
Ogiltig MQTT-data	JSON med value som sträng	Adaptern stoppar datan före XML-sändning
Trasig XML	Skicka XML med saknad sluttagg direkt till API:t	HTTP 400 och tidigare giltig mätning ligger kvar
Fel XML-innehåll	Skicka XML med saknat sensorId, fel enhet eller ogiltigt talformat	HTTP 400 och tidigare giltig mätning ligger kvar
Fel innehållstyp	Skicka giltig XML som text/plain	HTTP 415
Befintligt JSON-flöde	Skicka valid.json direkt till API:t som application/json	HTTP 201 eftersom JSON-stödet finns kvar

För att prova mottagarens XML-validering separat kan ni spara ett testfall i `reading.xml` och köra följande från demomappen i Windows:

```
curl.exe -i -H 'Content-Type: application/xml' --data-binary '@reading.xml'
http://127.0.0.1:8085/api/readings
.\build\Debug\consumer.exe
```

Detta kommando går direkt till API:t och testar därför inte MQTT-sträckan eller adaptern. I Linux används `curl` och `./build/consumer` med motsvarande argument. Kör även det ursprungliga integrationstestet för att kontrollera att JSON-stödet fortfarande fungerar, men komplettera med XML-fallen ovan eftersom det befintliga testet inte verifierar XML.

Frågor att använda som stöd

- Var i flödet sker formatomvandlingen och varför passar den i adaptern?
- Hur visar ni att mätningens betydelse är oförändrad trots olika representationer?
- Varför behöver API:t validera XML även när adaptern redan validerat JSON?
- Varför kan konsumenten fortsätta läsa JSON när POST-anropet använder XML?

Spara ett JSON-exempel, motsvarande XML-exempel och observerade testresultat. Den här valbara övningen kräver inte att gruppen byter representation i sitt befintliga IoT-case.

Ytterligare fördjupning

- Lägg till en dokumenterad tidsstämpel och visa när ett värde betraktas som gammalt
- Skissa hur en kontinuerlig MQTT-adapter skulle hantera API-avbrott och dubletter

Arbetsresultat

Spara en arkitekturskiss, ett giltigt exempel och testprotokollet. Du ska kunna visa hela flödet och lokalisera minst ett avsiktligt fel. Använd resultaten som stöd när du fortsätter med gruppens case.